COMBINED LITERATURE REVIEW

Explainable AI for Network Intrusion Detection Systems

Evidence synthesis, lessons learned, and implementation plan for the CSE 498R research project

Project proposal	A Quantitative Framework for Evaluating Explainable AI in Network Intrusion Detection Systems
Literature consolidated	Six papers reviewed across three source documents
Core focus	IDS performance, feature selection, explanation quality, and deployment feasibility
Prepared	24 July 2026

Purpose: bring the full literature review into one consistent document and turn the reviewed evidence into a concrete, reproducible plan for the proposed classifier-plus-explainer pipeline.

Executive Summary

The reviewed literature converges on a clear conclusion: high classification accuracy alone is not enough for a useful intrusion detection system. A practical IDS must detect minority attack classes, operate within latency constraints, and give explanations that are faithful to the model, stable for similar traffic, and understandable to analysts. The project proposal is therefore well positioned because it treats explanation quality as a measurable system property rather than as a decorative visualization.

Across the six papers, three recurring design ideas stand out. First, modern datasets such as CIC-IDS2017 and UNSW-NB15 should be preferred over legacy-only evaluation. Second, feature reduction can preserve or improve detection while reducing training and explanation cost. Third, SHAP and LIME should not be accepted at face value: their outputs require quantitative validation through perturbation, neighborhood consistency, and timing.

The recommended implementation is a leakage-safe experimental pipeline using Random Forest and a gradient-boosted tree model on CIC-IDS2017 and UNSW-NB15, with both binary and multiclass tasks. SHAP and LIME should be evaluated using precisely defined fidelity, stability, and speed metrics. Performance should be reported with macro-F1, per-class recall, false-positive rate, and confusion matrices rather than accuracy alone.

Recommended project decisions at a glance

Decision	Recommendation	Reason from literature
Data	Use CIC-IDS2017 and UNSW-NB15; keep dataset-specific preprocessing pipelines.	Modern traffic coverage and cross-dataset evidence reduce dependence on one benchmark.
Tasks	Run binary detection first, then multiclass attack identification.	Binary results can hide weak minority-class behavior that matters to analysts.
Models	Random Forest plus XGBoost/LightGBM-style gradient boosting.	Strong tabular baselines, native feature importance, and efficient TreeSHAP compatibility.
Explanations	TreeSHAP and LIME on the same stratified test instances.	Enables a fair comparison between theoretically consistent and locally intuitive methods.
Quality metrics	Deletion fidelity, local stability, and end-to-end latency.	Directly matches the proposal and addresses the largest gap in prior XAI-for-IDS work.
Feature reduction	Evaluate ranked feature subsets rather than choosing a fixed count arbitrarily.	Prior studies show large efficiency gains but often lack threshold sensitivity analysis.

Research gap addressed

Previous studies frequently show SHAP or LIME plots but do not establish whether the highlighted features truly control the prediction, whether similar flows receive similar explanations, or whether explanations are fast enough for operational use. This project can contribute a standardized, reproducible evaluation protocol that measures all three properties on two datasets and two classifiers.

1. Project Context and Research Questions

1.1 Problem statement

Machine-learning IDS models can generate accurate alerts while remaining difficult to trust and investigate. Without a defensible explanation, analysts may struggle to prioritize alerts, distinguish model errors from genuine attacks, or identify the traffic characteristics driving a decision. At scale, this contributes to alert fatigue and longer incident response.

1.2 Proposed system

The proposal defines an end-to-end pipeline consisting of labeled network-flow data, preprocessing, a tree-based classifier, local explanation generation with SHAP and LIME, and quantitative assessment of explanation fidelity, stability, and speed. CIC-IDS2017 and UNSW-NB15 are the primary datasets; Random Forest and gradient boosting are the primary classifiers.

1.3 Refined research questions

- **RQ1 – Predictive performance:** How accurately and consistently do the selected tree models detect normal and malicious traffic across binary and multiclass settings?
- **RQ2 – Fidelity:** When the features ranked most important by an explainer are perturbed or removed, how strongly does the model's original confidence decrease relative to controls?
- **RQ3 – Stability:** For nearby inputs that receive the same prediction, how similar are the corresponding ranked explanations?
- **RQ4 – Speed:** What latency and throughput costs do SHAP and LIME introduce, and which method fits offline, near-real-time, or analyst-on-demand use?
- **RQ5 – Robustness:** Do conclusions remain consistent across datasets, models, feature-set sizes, random seeds, and attack classes?

1.4 Expected contribution

The strongest contribution is not a claim that one explainer is universally best. It is a benchmark that makes the trade-offs visible and repeatable: explanation method, model family, feature count, quality score, and computation cost are evaluated together under a shared protocol.

2. Consolidated Review of the Six Papers

This section preserves the substantive content of the three supplied literature-review files. Each paper is presented with its objective, method, principal findings, limitations, and direct relevance to the proposed project.

2.1 Mane and Rao (2021) – Explainable deep-learning IDS

Objective and method

Mane and Rao address the black-box behavior of a deep neural network trained for binary intrusion detection on NSL-KDD. Their fully connected architecture used one-hot encoded categorical variables and min-max normalized inputs. They layered five explanation techniques over the classifier: SHAP for local and global contributions, LIME for local surrogates, CEM for contrastive explanations, ProtoDash for example-based explanations, and BRCG for human-readable Boolean rules.

Key findings

- The DNN reported 82.4% accuracy, 96.4% precision, 71.3% recall, and an F1 score of 0.820 on KDDTest+.
- SHAP highlighted features such as `same_srv_rate` and `dst_host_serror_rate` as influential.
- Five BRCG rules approximated the DNN at roughly 80% test accuracy, suggesting that a compact rule view can retain much of the model behavior.
- CEM showed that small changes to two or three features could flip predictions, demonstrating the value of counterfactual or perturbation-based inspection.
- Different explanation styles were useful for different audiences: feature contributions for technical analysis, examples and rules for broader interpretation.

Limitations and project relevance

The study lacks standardized explanation-quality metrics, uses binary classification only, does not validate utility with working analysts, and relies on the aging NSL-KDD dataset. SHAP scalability is also left unresolved. These gaps directly motivate this project's fidelity, stability, and speed benchmark on newer flow datasets.

2.2 Alazab et al. (2012) – Information-gain feature selection

Objective and method

Alazab et al. use information gain to rank the 41 NSL-KDD features and compare a full feature set with a reduced 12-feature set. A J48 decision tree is evaluated through 10-fold cross-validation over Normal, Probe, DoS, U2R, and R2L classes.

Key findings

- The reduced feature set achieved classification performance comparable to the full set, with weighted precision, recall, and F-measure around 0.983–0.996.
- Training and testing were approximately five times faster with the reduced set.
- Normal and DoS detection were strong, while U2R and R2L remained weaker, consistent with class imbalance.
- The reported classification rates compared favorably with several earlier IDS approaches.

Limitations and project relevance

Only one selector and one classifier were tested, the exact choice of 12 features was not justified through a feature-count curve, and neither streaming performance nor adversarial evasion was

assessed. The study nevertheless provides strong evidence that the proposed project should measure explanation cost across multiple feature-set sizes.

2.3 Khraisat et al. (2019) – Survey of IDS techniques, datasets, and challenges

Scope and method

This survey organizes IDS research by detection approach, machine-learning technique, feature selection, benchmark dataset, and evasion strategy. It contrasts signature-based systems, which recognize known patterns, with anomaly-based systems, which can detect novel deviations but commonly raise more false alarms.

Key findings

- Signature detection is dependable for known attacks but weak against zero-day behavior; anomaly detection broadens coverage at the cost of false positives.
- Reported model performance varies substantially across algorithms and experimental conditions, making direct comparison difficult.
- Legacy datasets such as DARPA and KDD99 remain common despite limited realism for modern traffic.
- Feature selection can improve both accuracy and processing time; one reviewed C4.5 result improved from 99.55% to 99.65% while reducing runtime from 2.76 to 0.84 seconds.
- Fragmentation, flooding, obfuscation, and encryption remain important evasion challenges.

Limitations and project relevance

As a survey, the paper offers no new implementation or reproducible code and gives limited attention to real-time operation or newer deep-learning developments. Its most useful contribution to this project is the broader warning that dataset age, evasion, and inconsistent evaluation can invalidate apparently high performance.

2.4 Ahmed et al. (2024) – Explainable hybrid ensemble IDS

Objective and method

Ahmed et al. propose HBB-RE, a hybrid of bagging, boosting, and residual-error correction, evaluated on CIC-IDS2017. The workflow includes SMOTE for imbalance, binary and multiclass evaluation, SHAP-based feature ranking, and SHAP/LIME visual explanations.

Key findings

- With SMOTE, the bagging-boosting stage reported 98.47% accuracy and F1 for binary classification and 92.75% multiclass accuracy.
- Without SMOTE, results remained strong but varied across the bagging-boosting and residual stages.
- Important CIC-IDS2017 features included `Flow IAT Max`, `Destination Port`, `Bwd Packet Length Std`, and `Init Win bytes forward`.
- SHAP summary, waterfall, and force plots made global patterns and individual decisions visible.
- Balancing especially improved the more difficult multiclass setting.

Limitations and project relevance

The architecture is computationally complex, uses only one dataset, does not report real-time explanation latency, and selects as many as 100 features without a clear redundancy analysis. It

demonstrates that accuracy and explanation plots can coexist, but also reinforces the need to measure cost and validate explanations rather than displaying them alone.

2.5 Kasongo and Sun (2020) – XGBoost-based feature selection

Objective and method

Kasongo and Sun apply an XGBoost feature-importance filter to UNSW-NB15, reducing 42 variables to 19. After encoding categorical variables and min-max scaling, they compare SVM, logistic regression, ANN, decision tree, and k-nearest neighbor models on binary and 10-class tasks.

Key findings

- Feature reduction improved several multiclass models; ANN test accuracy increased from 75.62% to 77.51% while using half as many hidden-layer neurons.
- The decision tree produced the best reported binary result at 90.85% test accuracy with 19 features.
- ANN was the strongest multiclass model at 77.51% accuracy, 79.50% precision, and 77.28% F1.
- Minority categories such as Worms and Shellcode were substantially harder to detect.
- SVM underperformed and logistic regression did not consistently benefit from feature selection.

Limitations and project relevance

The work does not address imbalance, cross-dataset generalization, live deployment, repeated trials, or explanation quality. The threshold producing 19 features is not fully justified. For this project, the lesson is to treat feature count as an experimental variable and to report uncertainty across seeds.

2.6 Gaspar, Silva, and Silva (2024) – Validating LIME and SHAP in an IoT IDS

Objective and method

Gaspar et al. train a three-layer MLP on a generated ADFA-LD system-call dataset. TF-IDF bigrams are reduced from 2,805 to 150 features using chi-squared selection. Kernel SHAP and LIME identify the ten most important features per instance. The study then tests explanations through feature removal or replacement and surveys 24 participants.

Key findings

- The MLP reported 93.71% accuracy, 98.06% precision, and 93.56% F1.
- Both explainers identified system calls that pushed errors in the wrong direction.
- Replacing `setsockopt` with `accept` reportedly changed 90% of selected false negatives into true positives.
- Targeted replacement caused much larger prediction shifts than simple feature removal.
- Participants generally preferred LIME's presentation, while more technical users favored SHAP's consistency; Kernel SHAP was much slower.

Limitations and project relevance

The evaluation uses one dataset, one MLP, and synthetic perturbations that may not represent feasible system-call traces. The survey sample is small, and the chosen KernelExplainer is not representative of optimized TreeSHAP. Even so, this is the clearest evidence among the reviewed papers that explanations should be tested by controlled perturbation.

3. Cross-Paper Synthesis: What We Learned

3.1 Detection performance must be interpreted carefully

Very high accuracy appears repeatedly, but the results are not directly comparable because datasets, preprocessing, task definitions, train-test protocols, and class distributions differ. Binary accuracy can also conceal failure on rare attack types. The project should therefore avoid declaring a model best from accuracy alone and should report macro-averaged and per-class metrics.

3.2 Feature selection is both a predictive and explainability decision

Alazab et al. and Kasongo and Sun show that fewer features can preserve or improve performance and reduce computation. For XAI, feature reduction has a second benefit: explanations become faster and easier to inspect. However, a fixed count such as 12, 19, or 100 should not be copied. The project should evaluate a curve of feature-set sizes and select a knee point based on predictive performance, explanation fidelity, and latency.

3.3 SHAP and LIME serve different operational needs

Property	SHAP	LIME	Project implication
Core idea	Additive feature attribution derived from Shapley values.	Sparse local surrogate fitted around one instance.	Use a shared output schema so rankings can be compared.
Strength	Consistent attribution framework; TreeSHAP is efficient for tree models.	Flexible, model-agnostic, and often easy to present locally.	Evaluate both on identical samples and hardware.
Risk	Background choice and explainer variant affect cost and meaning.	Sampling and discretization can make explanations unstable.	Record all explainer settings and random seeds.
Likely role	Offline investigation, global aggregation, and analyst-on-demand explanation.	Fast local explanation when a compact surrogate is acceptable.	Recommend by measured quality-latency trade-off, not by assumption.

3.4 Explanation visualization is not explanation validation

A convincing force plot or bar chart does not show that an explanation is faithful. The strongest reviewed validation idea is perturbation: if top-ranked features truly drive the prediction, modifying them should change model confidence more than modifying low-ranked or random features. Stability adds a second test by checking whether very similar inputs receive compatible explanations.

3.5 Class imbalance affects both the classifier and the explanations

Minority classes repeatedly show weaker detection. Oversampling can help, but applying SMOTE before the train-test split would leak information. Resampling must occur only inside training folds. Explanation quality should also be reported by class because a method may look stable overall while failing on rare attacks.

3.6 Real-time feasibility must be measured end to end

Several papers mention computational cost without reporting latency or throughput. This project should time preprocessing, prediction, and explanation separately, include warm-up and repeated runs, and report median plus tail latency. A realistic architecture may generate predictions for every flow but explanations only for high-risk or analyst-selected alerts.

4. Implementation Plan for the Proposed Project

4.1 System architecture

Stage	Implementation	Output/control
1. Ingest	Load CIC-IDS2017 and UNSW-NB15 from versioned local files; retain dataset identity.	Raw data manifest and checksums.
2. Clean	Remove duplicates, invalid labels, infinite values, and obvious identifiers; document every removed field.	Clean dataset plus audit log.
3. Split	Use stratified train/validation/test splits; consider time- or day-aware splitting for CIC-IDS2017.	Leakage-resistant partitions.
4. Transform	Fit imputers, encoders, and scalers on training data only, inside a pipeline.	Reusable preprocessing object.
5. Balance	Compare class weights with SMOTE applied only inside training folds.	Balanced-training variants.
6. Select features	Rank using training-only mutual information or model importance; test multiple top-k values.	Feature-count trade-off curve.
7. Train	Tune Random Forest and gradient boosting using stratified cross-validation.	Frozen model and configuration.
8. Explain	Run TreeSHAP and LIME on the same stratified evaluation sample.	Signed feature contributions and ranks.
9. Evaluate	Compute predictive metrics plus fidelity, stability, and speed with confidence intervals.	Comparable benchmark table.
10. Report	Generate plots, per-class analysis, reproducibility manifest, and deployment recommendation.	Paper-ready evidence package.

4.2 Dataset preparation details

- **Label harmonization:** Keep original labels for dataset-specific multiclass experiments and create a separate binary mapping for normal versus attack.
- **Duplicates:** Detect duplicates before splitting so identical flows cannot appear in both training and test sets.
- **Identifiers:** Review IP addresses, timestamps, flow IDs, and ports for leakage. Do not automatically remove ports; test whether they encode transferable behavior or dataset artifacts.
- **Missing and infinite values:** Replace only through a training-fitted strategy and record counts by feature.
- **Categorical variables:** Use an encoder with explicit handling for unseen categories in the test set.
- **Scaling:** Tree models do not require scaling, but LIME's distance calculations may. Maintain an explanation-space transformation that is consistent and reversible.
- **Sampling:** Preserve an untouched, naturally imbalanced test set. Any SMOTE or undersampling belongs inside training only.

4.3 Model training and baselines

Implement a simple baseline first: majority class, logistic regression, or a shallow decision tree. Then train Random Forest and gradient boosting with a compact, predeclared hyperparameter search.

Use the same folds for model comparisons. Save seeds, library versions, fitted preprocessing, feature order, and class mapping with each model artifact.

Metric family	Required measures	Why
Overall	Accuracy, balanced accuracy, macro-F1, weighted-F1	Shows both aggregate and imbalance-aware performance.
Attack detection	Per-class precision, recall, F1, support	Reveals failure on rare attack categories.
Operational	False-positive rate, false-negative rate, PR-AUC	Connects model behavior to alert load and missed attacks.
Diagnostic	Confusion matrix and calibration curve	Shows error structure and whether confidence is trustworthy.

4.4 Explanation generation protocol

- Choose a fixed, stratified evaluation set containing correct predictions, false positives, and false negatives from every class with sufficient support.
- For SHAP, use TreeExplainer for tree models and state the background/reference choice, output scale, and class index.
- For LIME, fix the kernel width, discretization policy, number of perturbation samples, number of displayed features, and seed.
- Store explanations as signed attribution vectors in the original feature order, together with the model probability and metadata.
- Separate explanation generation from plotting so metrics are computed from raw values rather than images.

5. Quantitative Explanation Metrics

5.1 Fidelity: top-k deletion or masking

For each test instance, rank features by absolute attribution. Replace the top-k features using a valid reference value, such as a training-set median or a conditional/background draw, recompute the predicted probability for the original class, and measure the drop. A stronger faithful explanation should cause a larger drop than random or bottom-ranked controls.

Primary fidelity score: mean probability drop across k in a predeclared set, such as $k = 1, 3, 5, 10$. Secondary outcomes: prediction flip rate and area over the deletion curve. Report results by model, explainer, dataset, and class.

Important caution: independently replacing network-flow features can create impossible traffic records. To reduce this threat, compare several baselines and add plausibility checks. Where possible, sample replacements from same-class or nearest training neighbors rather than assigning arbitrary zeros.

5.2 Stability: explanation consistency for similar inputs

Construct local pairs from test instances that share the same predicted class and are close in normalized feature space. Compare their explanation vectors using cosine similarity and their top-k rankings using Jaccard overlap or rank correlation. A normalized stability score can relate explanation change to input change, with a small epsilon to prevent division by zero.

- Use nearest neighbors from the held-out set, not synthetic points only.
- Require the original prediction to remain unchanged; otherwise explanation changes may be appropriate.
- Report the distribution, not only the mean, and stratify by attack class.
- Repeat LIME with different seeds on the same instance to measure stochastic self-stability separately.

5.3 Speed: latency and throughput

Measure wall-clock explanation time on a fixed machine after warm-up. Report median, 90th, 95th, and 99th percentile latency, plus explanations per second. Separate single-instance analyst-on-demand timing from batched offline timing. Record CPU/GPU, RAM, software versions, number of LIME samples, SHAP explainer variant, background size, and feature count.

5.4 Statistical comparison

- Use paired comparisons because SHAP and LIME explain the same instances.
- Bootstrap the instance-level metric differences to obtain 95% confidence intervals.
- Report effect sizes and confidence intervals in addition to p-values.
- Correct for repeated testing when comparing many k values, classes, or configurations.
- Predeclare the primary metric and decision rule to reduce selective reporting.

5.5 Proposed benchmark output

Dataset	Model	Task	Explainer	Fidelity	Stability	Median ms	P95 ms	Macro-F1
CIC-IDS2017	Random Forest	Binary	TreeSHAP	to measure	to measure	to measure	to measure	model result
CIC-IDS2017	Random Forest	Binary	LIME	to measure	to measure	to measure	to measure	same model
UNSW-NB15	Gradient Boosting	Multiclass	TreeSHAP	to measure	to measure	to measure	to measure	model result
UNSW-NB15	Gradient Boosting	Multiclass	LIME	to measure	to measure	to measure	to measure	same model

Template only; no experimental values are invented in this literature synthesis.

6. Experimental Matrix and Ablation Studies

6.1 Minimum viable experiment

Factor	Levels
Datasets	CIC-IDS2017; UNSW-NB15
Tasks	Binary; multiclass
Models	Random Forest; gradient-boosted trees
Explainers	TreeSHAP; LIME
Feature sets	All features; selected top-k feature set
Balancing	Class weights; training-fold SMOTE where appropriate
Seeds	At least five for stochastic components

6.2 High-value ablations

- **Feature-count ablation:** Compare all features with top 5, 10, 20, 30, and an automatically selected knee point.
- **Perturbation baseline ablation:** Zero, median, marginal sample, and nearest-neighbor replacement.
- **LIME sampling ablation:** Vary the number of perturbation samples to quantify the stability-speed trade-off.
- **SHAP background ablation:** Vary background size or selection strategy where the chosen explainer requires it.
- **Balancing ablation:** No balancing, class weights, and fold-local SMOTE.
- **Error-type analysis:** Compare explanations for true positives, false positives, true negatives, and false negatives.

6.3 Success criteria

- The classifier must outperform simple baselines on macro-F1 and show acceptable per-class recall.
- Top-ranked feature perturbation must produce a materially larger confidence drop than random and bottom-ranked controls.
- Stability must remain high for same-class local neighbors and should not collapse under repeated LIME seeds.
- Timing must support a clearly stated use mode: offline batch, analyst-on-demand, or near-real-time.
- All results must be reproducible from a configuration file and recorded random seeds.

7. Practical Deployment Recommendation

The literature does not support a universal claim that SHAP belongs offline and LIME belongs at the edge; actual performance depends on the model and explainer implementation. For tree models, TreeSHAP can be very fast. The project should therefore base deployment guidance on measured results.

Operating mode	Recommended behavior
Every network flow	Run preprocessing and classification only; log compact model outputs and feature values needed for later explanation.
High-risk alert	Generate a local explanation immediately if P95 latency meets the alerting budget.
Analyst investigation	Generate richer SHAP/LIME views, nearest examples, and controlled what-if analysis on demand.
Offline governance	Aggregate SHAP values, inspect drift, compare class-specific fidelity and stability, and audit recurring false positives.

A useful analyst-facing explanation should include the predicted class and calibrated confidence, the top contributing features with direction, observed values and understandable units, a short plain-language interpretation, similar reference flows, and an explicit warning that feature attribution is not proof of causality.

8. Risks, Validity Threats, and Mitigations

Risk	Why it matters	Mitigation
Data leakage	Flow identifiers, duplicates, or preprocessing fitted before splitting can inflate results.	Split before fitting transformations; deduplicate across partitions; audit suspiciously dominant features.
Dataset artifacts	A model may learn capture environment rather than attack behavior.	Use two datasets, temporal/day-aware splits where possible, and cross-dataset discussion.
Class imbalance	Accuracy can hide missed rare attacks.	Use macro-F1, per-class recall, PR-AUC, class weights, and fold-local balancing.
Unrealistic perturbations	Fidelity tests may create impossible flows.	Use multiple reference strategies, range checks, and neighbor-based replacements.
Explainer randomness	LIME rankings can vary across runs.	Fix and vary seeds deliberately; report self-stability.
Feature dependence	Independent attribution or masking can misrepresent correlated features.	Analyze correlated groups and discuss conditional versus marginal assumptions.
Latency bias	One timing run is unreliable.	Warm up, repeat, report distributions, and record hardware/software.
Human usefulness	A mathematically faithful explanation may still confuse analysts.	If scope permits, add a small task-based usability study; otherwise state this as future work.

9. Eight-Week Execution Plan

Week	Work package	Deliverable
1	Finalize protocol, acquire data, create data dictionary and repository structure.	Data manifest and preregistered experiment matrix.
2	Clean, deduplicate, label-map, split, and implement preprocessing pipelines.	Validated dataset report and leakage checks.
3	Train baselines, Random Forest, and gradient boosting; tune on training folds.	Model comparison with predictive metrics.

Week	Work package	Deliverable
4	Freeze models; integrate TreeSHAP and LIME; standardize explanation storage.	Reproducible explanation generator.
5	Implement fidelity and stability metrics with random/bottom-feature controls.	Metric tests and pilot results.
6	Benchmark speed; run full matrix and key ablations.	Complete raw results and confidence intervals.
7	Analyze classes and failure cases; create figures and deployment guidance.	Paper-ready tables, plots, and discussion.
8	Re-run from a clean environment, package code, and finalize the paper.	Reproducibility bundle and submission draft.

10. Final Takeaway

The combined literature supports the proposal's central direction and sharpens it. The project should not merely attach SHAP and LIME to accurate classifiers. It should test whether explanations change when the model's true decision drivers are disturbed, remain consistent around similar traffic, and arrive quickly enough for a declared operational setting. Coupling those measurements with modern datasets, leakage-safe preprocessing, minority-class analysis, and feature-count ablations would produce a credible and practically useful XAI-for-IDS benchmark.

References Reviewed

Bibliographic entries below are limited to the information available in the supplied review documents. Before academic submission, the team should replace these with complete citations copied from the original papers in the required style.

1. Ahmed et al. (2024). *Explainable AI-Based Innovative Hybrid Ensemble Model for Intrusion Detection*. Scientific Reports, 14:30532.
2. Alazab et al. (2012). Study of information-gain feature selection and J48 classification for intrusion detection on NSL-KDD.
3. Gaspar, D., Silva, P., and Silva, C. (2024). Study applying and validating LIME and SHAP for an MLP-based IoT intrusion detector using ADFA-LD.
4. Kasongo, S. M., and Sun, Y. (2020). Study of XGBoost filter-based feature selection and machine-learning classifiers on UNSW-NB15.
5. Khraisat et al. (2019). *Survey of Intrusion Detection Systems: Techniques, Datasets and Challenges*. Cybersecurity, 2(20).
6. Mane and Rao (2021). Study of a multi-method explainable-AI framework for a deep-learning intrusion detector on NSL-KDD.

Appendix A. Compact Glossary

Term	Meaning in this project
Accuracy	Fraction of all predictions that are correct; can be misleading under class imbalance.
Precision	Of alerts predicted as an attack, the fraction that are truly attacks.

Term	Meaning in this project
Recall	Of actual attacks, the fraction detected by the IDS.
Macro-F1	Average F1 across classes with equal class weight; highlights minority-class weakness.
False positive	Normal traffic incorrectly flagged as an attack; contributes to analyst alert fatigue.
Feature	A measured property of a network flow, such as duration, packet length, or inter-arrival time.
SHAP	An additive feature-attribution framework based on Shapley-value ideas.
LIME	A local surrogate method that approximates a model near one input.
Fidelity	The extent to which an explanation reflects the model's actual decision behavior.
Stability	The consistency of explanations for similar inputs or repeated runs.
SMOTE	A training-time oversampling method that synthesizes minority-class examples.
Data leakage	Use of information during training that would not legitimately be available for new predictions.